

COURSE CODE/TITLE:	CPE 212	SECTION:	CPE31S2
DATE PERFORMED:	July 16, 2026	DATE SUBMITTED:	July 30, 2026
NAME:	Mark David M. Loterte	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 SSH Key-Based Authentication and GIT Setup

1. PROCEDURE OUTPUT

Part 1

Task 1: Create an SSH Key Pair for User Authentication

- The simplest way to generate a key pair is to run ssh-keygen without arguments. In this case, it will prompt for the file in which to store keys. First, the tool asked where to save the file. SSH keys for user authentication are usually stored in the users .ssh directory under the home directory. However, in enterprise environments, the location is often different. The default key file name depends on the algorithm, in this case id_rsa when using the default RSA algorithm. It could also be, for example, id_dsa or id_ecdsa.
- Issue the command ssh-keygen -t rsa -b 4096. The algorithm is selected using the -t option and key size using the -b option.

```
mdmlot@workstation:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/mdmlot/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/mdmlot/.ssh/id_rsa
Your public key has been saved in /home/mdmlot/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:p9EXnnRp6CkXrEac70VQM2iMGCrFXJxuODy9Tf9B3Hg mdmlot@workstation
The key's randomart image is:
+----[RSA 4096]----+
|  o.o+  o.o+  |
| .o+..o+o  +  |
| ...+  +.*.=o  |
| . = +o.* 0+ E |
| +S+*.O...    |
| . = . . .    |
| .   . . .    |
| .             |
+-----[SHA256]-----+

mdmlot@workstation:~$ ls -la .ssh
total 24
drwx----- 2 mdmlot mdmlot 4096 Jul 16 09:53 .
drwxr-x--- 16 mdmlot mdmlot 4096 Jul  9 10:24 ..
-rw----- 1 mdmlot mdmlot 3381 Jul 16 09:53 id_rsa
-rw-r--r-- 1 mdmlot mdmlot 744 Jul 16 09:53 id_rsa.pub
-rw----- 1 mdmlot mdmlot 2240 Jul 16 09:24 known_hosts
-rw----- 1 mdmlot mdmlot 1120 Jul 16 09:12 known_hosts.old
mdmlot@workstation:~$
```

Task 2: Copying the Public Key to the remote servers

- To use public key authentication, the public key must be copied to a server and installed in an authorized_keys file. This can be conveniently done using the ssh-copy-id tool.
- Issue the command similar to this: ssh-copy-id -i ~/.ssh/id_rsa user@host

```
mdmlot@workstation:~$ ssh-copy-id -i ~/.ssh/id_rsa mdmlot@server1
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/mdmlot/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
mdmlot@server1's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'mdmlot@server1'"
and check to make sure that only the key(s) you wanted were added.

mdmlot@workstation:~$
```

```
mdmlot@workstation:~$ ssh-copy-id -i ~/.ssh/id_rsa mdmlot@server2
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/mdmlot/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
mdmlot@server2's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'mdmlot@server2'"
and check to make sure that only the key(s) you wanted were added.
```

3. Once the public key has been configured on the server, the server will allow any connecting user that has the private key to log in. During the login process, the client proves possession of the private key by digitally signing the key exchange.

4. On the local machine, verify that you can SSH with Server 1 and Server 2. What did you notice? Did the connection ask for a password? If not, why?

```
mdmlot@workstation:~$ ssh mdmlot@server1
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-134-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Thu Jul 16 09:58:34 2026 from 192.168.56.112
mdmlot@server1:~$
logout
Connection to server1 closed.
```

```
mdnlot@workstation:~$ ssh mdnlot@server2
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-134-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Thu Jul 16 09:24:52 2026 from 192.168.56.112
mdnlot@server2:~$
logout
Connection to server2 closed.
mdnlot@workstation:~$
```

I noticed that I was no longer required to enter the password of either server1 and server2. This was because the workstation's public cryptographic key was copied via ssh-copy-id to the server's list of authorized users.

Reflections:

Answer the following:

1. How will you describe the ssh-program? What does it do?

I would say that the SSH program is a useful program in order to securely and quickly connect from one end device in a network to another. Through this secure tunneled connection, we are able to use the terminal of the remote device in order to control many aspects such as active processes and installed files.

2. How do you know that you already installed the public key to the remote servers?

You will know if the public key is installed if you check ~/.ssh/authorized_keys in the remote server or if you're no longer required to log in to connect to said server.

Part 2:

Task 3: Set up the Git Repository

1. On the local machine, verify the version of your git using the command which git. If a directory of git is displayed, then you don't need to install git. Otherwise, to install git, use the following command: sudo apt install git

```
mdmlot@workstation:~$ which git
mdmlot@workstation:~$ sudo apt install git
[sudo] password for mdmlot:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libsmbios-c2
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  git-man liberror-perl
Suggested packages:
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb
  git-cvs git-mediawiki git-svn
The following NEW packages will be installed:
  git git-man liberror-perl
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 4,154 kB of archives.
After this operation, 21.0 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://ph.archive.ubuntu.com/ubuntu jammy/main amd64 liberror-perl all 0
17029-1 [26.5 kB]
Get:2 http://ph.archive.ubuntu.com/ubuntu jammy-updates/main amd64 git-man all
1:2.34.1-1ubuntu1.17 [954 kB]
Get:3 http://ph.archive.ubuntu.com/ubuntu jammy-updates/main amd64 git amd64 1
2.34.1-1ubuntu1.17 [3,174 kB]
85% [3 git 2,750 kB/3,174 kB 87%] 584 kB/s 0s
```

2. After the installation, issue the command which git again. The directory of git is usually installed in this location: user/bin/git.

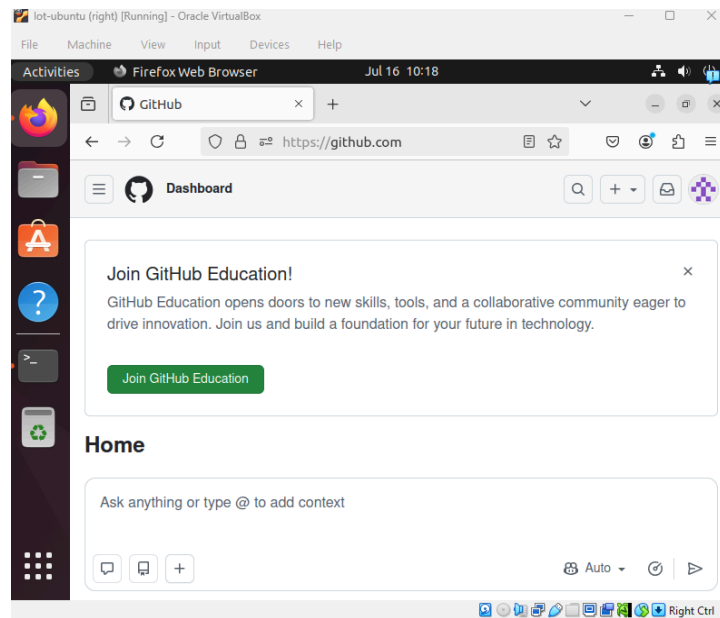
```
mdmlot@workstation:~$ which git
/usr/bin/git
```

3. The version of git installed in your device is the latest. Try issuing the command git --version to know the version installed.

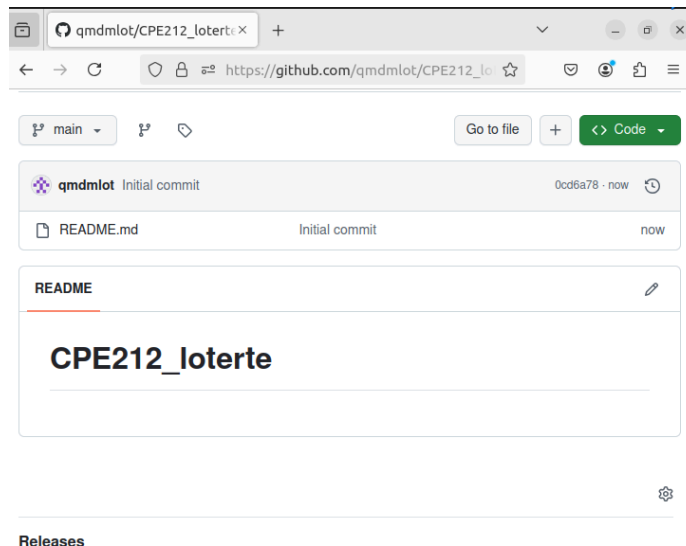
```
mdmlot@workstation:~$ git --version
git version 2.34.1
```

4. Using the browser in the local machine, go to www.github.com.

5. Sign up in case you don't have an account yet. Otherwise, login to your GitHub account.



a. Create a new repository and name it as CPE232_yourname. Check Add a README file and click Create repository.



b. Create a new SSH key on GitHub. Go to your profile's settings and click SSH and GPG keys. If there is an existing key, make sure to delete it. To create new SSH keys, click New SSH Key. Write CPE232 key as the title of the key.

c. On the local machine's terminal, issue the command `cat ~/.ssh/id_rsa.pub` and copy the public key. Paste it on the GitHub key and press Add SSH key.

This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.

Authentication keys



CPE212 key
SHA256:p9EXnnRp6CkXrEac70VQM21MGCrFXJxu0Dy9Tf9B3Hg
Added on Jul 30, 2026
Never used — Read/write

Delete

d. Clone the repository that you created. In doing this, you need to get the link from GitHub. Browse to your repository as shown below. Click on the Code drop down menu. Select SSH and copy the link.

Initial commit

Local Codespaces

Clone

HTTPS SSH GitHub CLI

git@github.com:qmdmlot/CPE212_loterte.git

Use a password-protected SSH key.

Open in GitHub Copilot app

Download ZIP

e. Issue the command git clone followed by the copied link. For example, git clone git@github.com:jvtaylor-cpe/CPE232_yourname.git. When prompted to continue connecting, type yes and press enter.

```
mdmlot@workstation: ~  
mdmlot@workstation:~$ git clone git@github.com:qmdmlot/CPE212_loterte.git  
Cloning into 'CPE212_loterte'...  
remote: Enumerating objects: 3, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Receiving objects: 100% (3/3), done.  
mdmlot@workstation:~$
```

f. To verify that you have cloned the GitHub repository, issue the command ls. Observe that you have the CPE232_yourname in the list of your directories. Use CD command to go to that directory and LS command to see the file README.md.

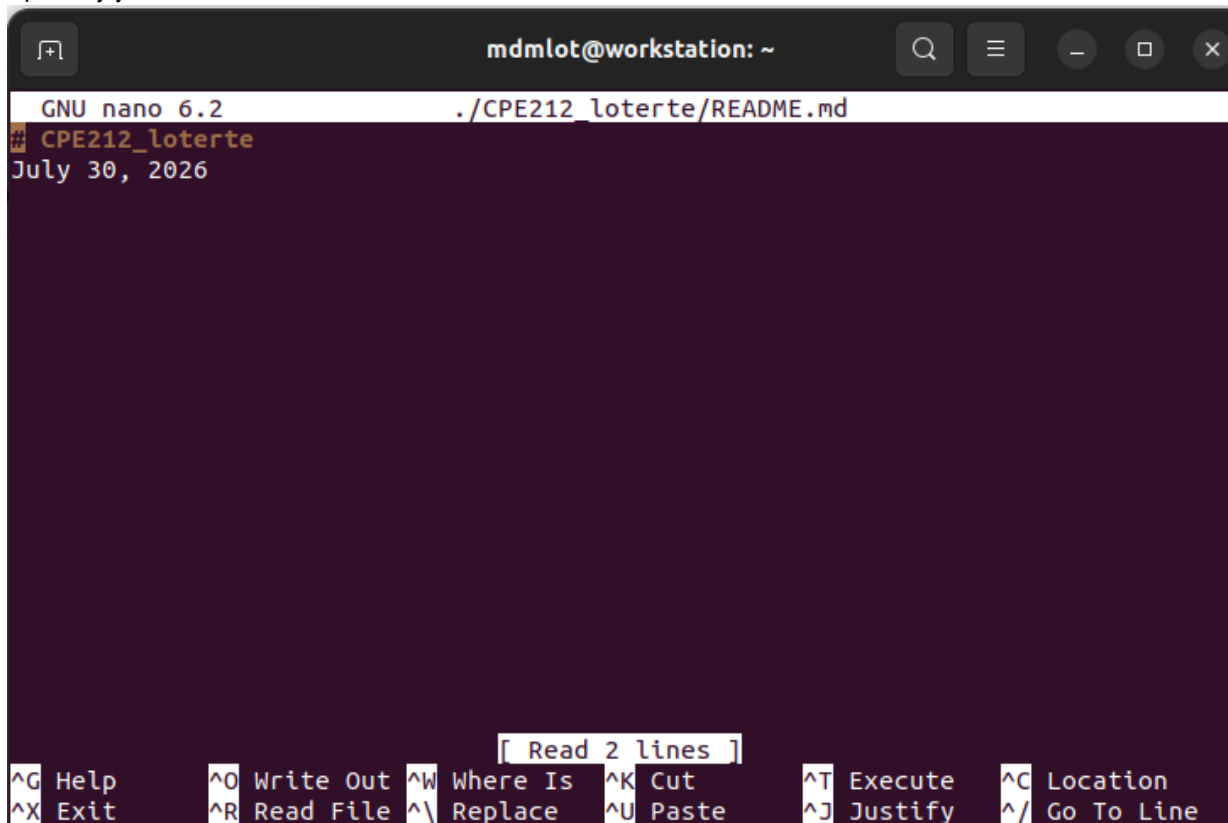
```
mdm1ot@workstation:~$ ls
CPE212_loterte Desktop Documents Downloads Music Pictures Public snap Templates Videos
mdm1ot@workstation:~$ cd CPE212_loterte
mdm1ot@workstation:~/CPE212_loterte$ ls
README.md
mdm1ot@workstation:~/CPE212_loterte$
```

g. Use the following commands to personalize your git.

- git config --global user.name "Your Name"
- git config --global user.email yourname@email.com
- Verify that you have personalized the config file using the command cat ~/.gitconfig

```
mdm1ot@workstation:~/CPE212_loterte$ git config --global user.name "Mark David Loterte"
mdm1ot@workstation:~/CPE212_loterte$ git config --global user.email qmdm1oterte@tip.edu.ph
mdm1ot@workstation:~/CPE212_loterte$ cd ..
mdm1ot@workstation:~$ cat ~/.gitconfig
[user]
  name = Mark David Loterte
  email = qmdm1oterte@tip.edu.ph
mdm1ot@workstation:~$
```

h. Edit the README.md file using nano command. Provide any information on the markdown file pertaining to the repository you created. Make sure to write out or save the file and exit.



```
mdm1ot@workstation: ~
GNU nano 6.2 ./CPE212_loterte/README.md
CPE212_loterte
July 30, 2026

[ Read 2 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut      ^T Execute  ^C Location
^X Exit      ^R Read File ^_ Replace   ^U Paste    ^J Justify  ^_ Go To Line
```

i. Use the git status command to display the state of the working directory and the staging area. This command

shows which changes have been staged, which haven't, and which files aren't being tracked by Git. Status output does not show any information regarding the committed project history. What is the result of issuing this command?

```
mdmlot@workstation:~/CPE212_loterte$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
mdmlot@workstation:~/CPE212_loterte$
```

j. Use the command `git add README.md` to add the file into the staging area.

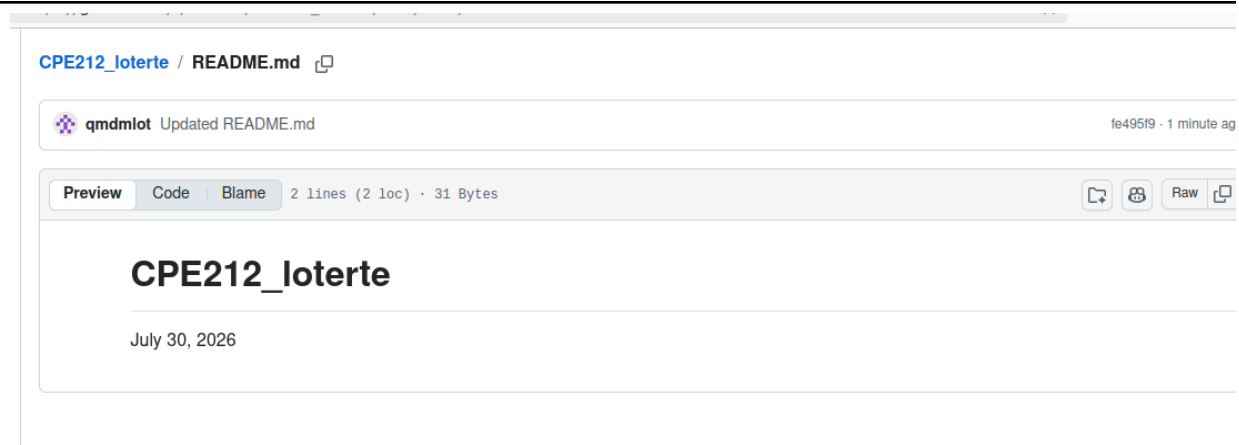
k. Use the `git commit -m "your message"` to create a snapshot of the staged changes along the timeline of the Git projects history. The use of this command is required to select the changes that will be staged for the next commit.

```
mdmlot@workstation:~/CPE212_loterte$ git add README.md
mdmlot@workstation:~/CPE212_loterte$ git commit -m "Updating README.md"
[main 82594bb] Updating README.md
 1 file changed, 2 insertions(+), 1 deletion(-)
mdmlot@workstation:~/CPE212_loterte$
```

l. Use the command `git push <remote><branch>` to upload the local repository content to GitHub repository. Pushing means to transfer commits from the local repository to the remote repository. As an example, you may issue `git push origin main`.

```
mdmlot@workstation:~/CPE212_loterte$ git add README.md
mdmlot@workstation:~/CPE212_loterte$ git commit -m "Updated README.md"
[main fe495f9] Updated README.md
 1 file changed, 2 insertions(+), 1 deletion(-)
mdmlot@workstation:~/CPE212_loterte$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 290 bytes | 290.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:qmdmlot/CPE212_loterte.git
   0cd6a78..fe495f9  main -> main
mdmlot@workstation:~/CPE212_loterte$
```

m. On the GitHub repository, verify that the changes have been made to README.md by refreshing the page. Describe the README.md file. You can notice the how long was the last commit. It should be some minutes ago and the message you typed on the `git commit` command should be there. Also, the README.md file should have been edited according to the text you wrote.



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Conclusion:

I conclude that ssh is a valuable tool in order to remotely connect to other end devices for configuration and other management purposes without having to access the actual physical computer or server directly. Git and Github are also useful tools which will allow me to connect online repositories to my PC for local work and also allow me to actually collaborate and contribute to software development much more easily. Putting more practice into using these tools will allow me to streamline my workflow in programming and system administration much more efficiently.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.